

Exp No.: 1 Setting up Hyperledger Fabric Development Environment

Aim

To install and configure the Hyperledger Fabric development environment on a Linux system using Docker, Go, and Fabric binaries, create a blockchain test network, and establish a communication channel between peer organizations.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Go (Version 1.20 or later)
- Git
- Curl
- Hyperledger Fabric Samples
- Hyperledger Fabric Docker Images

Hardware Requirements

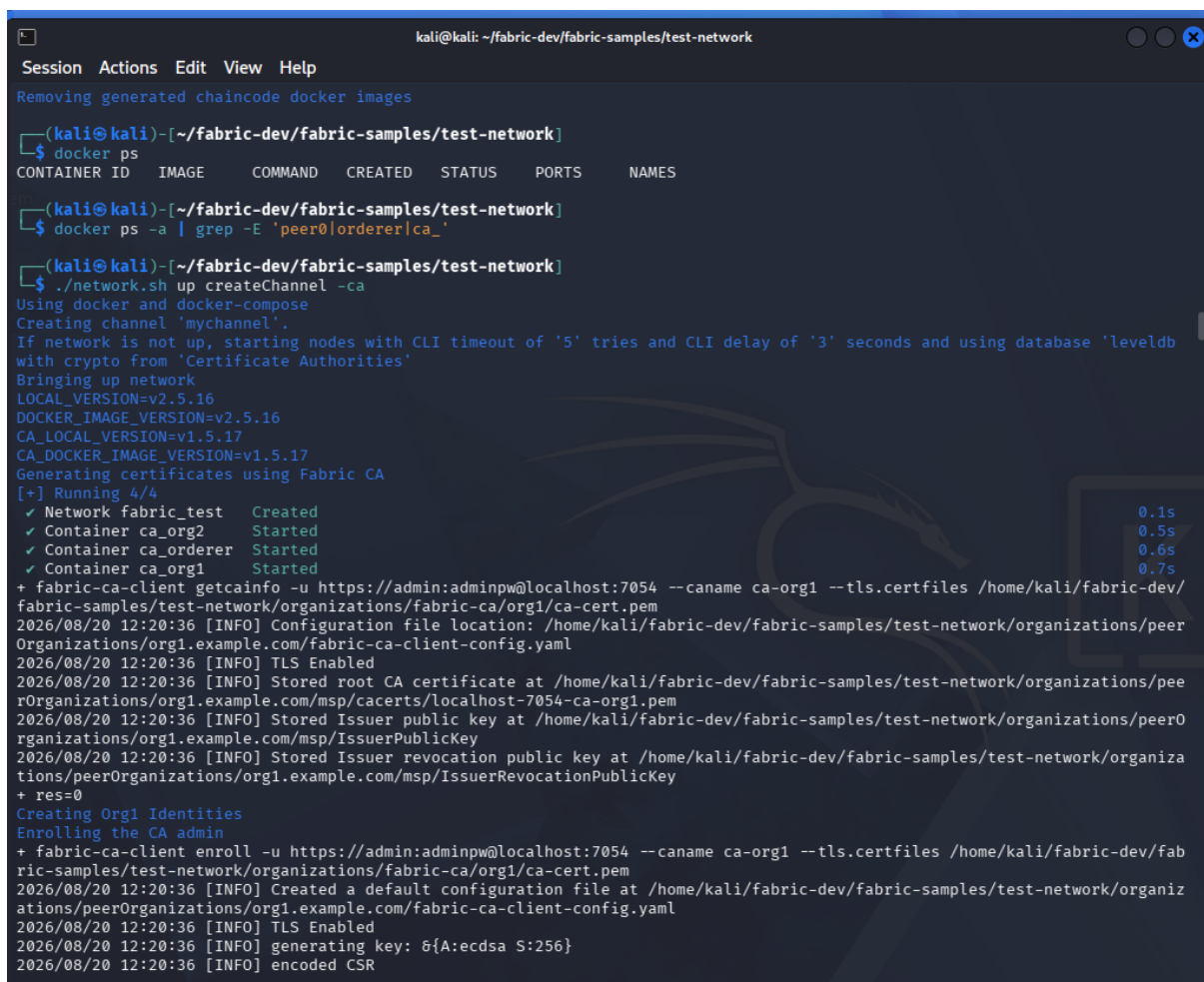
- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB free disk space
- Internet Connection

Procedure

Step 1: Install prerequisites (git, curl, jq, Docker, Docker Compose, Go)

Step 2: Download Hyperledger Fabric samples and Docker images

Step 3: Start the test network and create a channel



```
kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
Removing generated chaincode docker images

(kali@kali)-~/fabric-dev/fabric-samples/test-network
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
(kali@kali)-~/fabric-dev/fabric-samples/test-network
$ docker ps -a | grep -E 'peer0|orderer|ca_'

(kali@kali)-~/fabric-dev/fabric-samples/test-network
$ ./network.sh up createChannel -ca
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb
with crypto from 'Certificate Authorities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
[+] Running 4/4
  ✓ Network fabric_test      Created      0.1s
  ✓ Container ca_org2        Started      0.5s
  ✓ Container ca_orderer     Started      0.6s
  ✓ Container ca_org1        Started      0.7s
+ fabric-ca-client getcainfo -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/kali/fabric-dev/
fabric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2026/08/20 12:20:36 [INFO] Configuration file location: /home/kali/fabric-dev/fabric-samples/test-network/organizations/peer
Organizations/org1.example.com/fabric-ca-client-config.yaml
2026/08/20 12:20:36 [INFO] TLS Enabled
2026/08/20 12:20:36 [INFO] Stored root CA certificate at /home/kali/fabric-dev/fabric-samples/test-network/organizations/pee
rOrganizations/org1.example.com/msp/cacerts/localhost-7054-ca-org1.pem
2026/08/20 12:20:36 [INFO] Stored Issuer public key at /home/kali/fabric-dev/fabric-samples/test-network/organizations/peerO
rganizations/org1.example.com/msp/IssuerPublicKey
2026/08/20 12:20:36 [INFO] Stored Issuer revocation public key at /home/kali/fabric-dev/fabric-samples/test-network/organiza
tions/peerOrganizations/org1.example.com/msp/IssuerRevocationPublicKey
+ res=0
Creating Org1 Identities
Enrolling the CA admin
+ fabric-ca-client enroll -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/kali/fabric-dev/fab
ric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2026/08/20 12:20:36 [INFO] Created a default configuration file at /home/kali/fabric-dev/fabric-samples/test-network/organiz
ations/peerOrganizations/org1.example.com/fabric-ca-client-config.yaml
2026/08/20 12:20:36 [INFO] TLS Enabled
2026/08/20 12:20:36 [INFO] generating key: 5{A:ecdsa S:256}
2026/08/20 12:20:36 [INFO] encoded CSR
```

Figure 1: Starting the Fabric test network with Certificate Authorities and creating the channel

Step 5: Deploy chaincode on the channel

Figure 2: Anchor peer configuration, channel update, and chaincode deployment

Step 6: Set environment variables and verify chaincode installation

Step 7: Query the ledger

```
kali@kali: ~/fabric-dev/fabric-samples/test-network

Session Actions Edit View Help

/home/kali/fabric-dev/fabric-samples/test-network-k8s/config/org1/core.yaml
/home/kali/fabric-dev/fabric-samples/config/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/addOrg3/compose/podman/peercfg/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/addOrg3/compose/docker/peercfg/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/compose/podman/peercfg/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/compose/docker/peercfg/core.yaml

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ export CORE_PEER_LOCALMSPID=Org1MSP
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_ADDRESS=localhost:7051
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ echo $FABRIC_CFG_PATH
echo $CORE_PEER_LOCALMSPID
echo $CORE_PEER_ADDRESS
/home/kali/fabric-dev/fabric-samples/config
Org1MSP
localhost:7051

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629, Label: basic_1.0

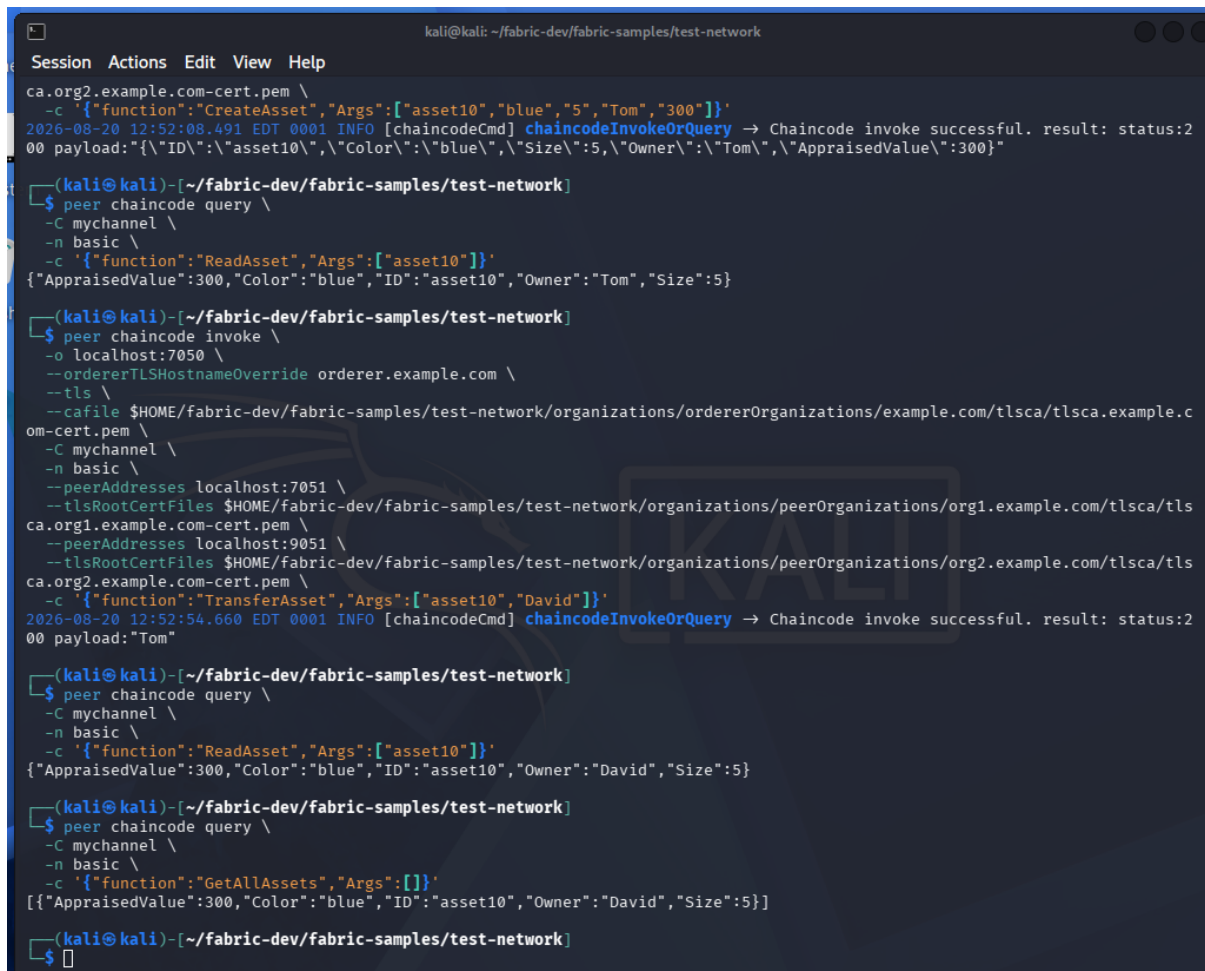
(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer lifecycle chaincode querycommitted \
--channelID mychannel \
--name basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-c mychannel \
-n basic \
-c '{"function": "GetAllAssets", "Args": []}'
[]

(kali@kali)~/fabric-dev/fabric-samples/test-network
```

Figure 3: Setting environment variables, verifying chaincode installation and querying the ledger

Step 8: Invoke chaincode — Create, Read, and Transfer assets



```

kali@kali: ~/fabric-dev/fabric-samples/test-network

Session Actions Edit View Help
ca.org2.example.com-cert.pem \
-c '{"function": "CreateAsset", "Args": ["asset10", "blue", "5", "Tom", "300"]}'
2026-08-20 12:52:08.491 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:2
00 payload:{"ID":"asset10","Color":"blue","Size":5,"Owner":"Tom","AppraisedValue":300}

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function": "ReadAsset", "Args": ["asset10"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset10","Owner":"Tom","Size":5}

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.c
om-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tls
ca.org1.example.com-cert.pem \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tls
ca.org2.example.com-cert.pem \
-c '{"function": "TransferAsset", "Args": ["asset10", "David"]}'
2026-08-20 12:52:54.660 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:2
00 payload:"Tom"

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function": "ReadAsset", "Args": ["asset10"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset10","Owner":"David","Size":5}

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function": "GetAllAssets", "Args": []}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset10","Owner":"David","Size":5}]

(kali@kali)~/fabric-dev/fabric-samples/test-network
$

```

Figure 4: Creating an asset, reading it, transferring ownership, and querying all assets

Result

The Hyperledger Fabric development environment was successfully installed and configured. The required Fabric binaries and Docker images were installed, the test network was created successfully, the mychannel application channel was generated, both peer organizations joined the channel, and the anchor peers were configured successfully. The chaincode was deployed, and asset creation, reading, transfer, and querying operations were performed successfully. The development environment is now ready for deploying chaincode (smart contracts) and developing blockchain applications.